

Homework - Neural networks - Part F (30 points)

ALCOVE: An exemplar-based connectionist model of category learning

by Brenden Lake

In this part, you will help implement ALCOVE (Kruschke, 1992), an influential exemplar-based neural network model of human category learning. Unlike networks that learn abstract feature detectors in a hidden layer, ALCOVE stores every training item as an **exemplar** and classifies new items by their similarity to those stored exemplars. ALCOVE learns two things: how much attention to pay to each stimulus dimension, and how strongly each exemplar is associated with each category.

Kruschke, J. K. (1992). ALCOVE: An exemplar-based connectionist model of category learning. *Psychological Review*, 99(1), 22–44.

We will test ALCOVE on the classic benchmark from Shepard, Hovland, and Jenkins (1961; hereafter SHJ), a study that has influenced concept learning theory for over 60 years. SHJ used 8 stimuli that vary along 3 binary dimensions (e.g., shape, size, and color), and asked people to learn 6 different ways of sorting these 8 stimuli into two categories of 4. The 6 **problem types** differ in how many dimensions are relevant and how they interact:

- **Type I:** only 1 dimension is relevant (e.g., all category A items share one feature value).
- **Type II:** 2 dimensions are relevant, combined via XOR (this is the classic "exclusive or" problem).
- **Types III, IV, V:** 3 dimensions are relevant, but one dimension is *mostly* predictive with a few exceptions.
- **Type VI:** all 3 dimensions are relevant and none is more predictive than another (the hardest, most "arbitrary" partition).

People reliably learn these problems in the order $I < II < III \approx IV \approx V < VI$ (from easiest to hardest), even though every problem type asks for exactly the same number of stimulus-response pairs to be learned. Explaining this ordering is a classic challenge for models of category learning, and it is exactly what ALCOVE was built to address through its selective attention mechanism.

The 8 stimuli and 6 problem types

Each stimulus is a length-3 binary vector (one bit per dimension). The label files below give the category assignment ('A' or 'B') of each of the 8 stimuli, for each of the 6 problem types. Run the cell below to load and print them.

```
In [5]: # Packages we need
from __future__ import print_function
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib
%matplotlib inline
import matplotlib.pyplot as plt
```

```
In [6]: CLASS_A, CLASS_B = 1., 0.

def load_shj():
    # Load the 8 SHJ stimuli (3 binary dimensions each) and their category
    # labels for each of the 6 SHJ problem types.
    #
    # Output
    #   X : [8 x 3 [tensor] the 8 stimuli, one per row, used as ALCOVE's ex
    #   y_list : list of 6 [8 tensor], category label (1/0) for each stimulu
    #           one tensor per problem type
    stimuli = np.genfromtxt('data/shj_stimuli.txt', delimiter=',', dtype=fl
    labels = np.genfromtxt('data/shj_labels.txt', delimiter=',', dtype=str)
    X = torch.tensor(stimuli).float()
    y_list = []
    for mytype in range(labels.shape[0]):
        y = np.where(labels[mytype] == 'A', CLASS_A, CLASS_B)
        y_list.append(torch.tensor(y).float())
    return X, y_list

X, y_list = load_shj()
print('Stimuli (8 exemplars x 3 binary dimensions):')
print(X.numpy().astype(int))
print('')
for t in range(6):
    print('Type ' + str(t+1) + ' labels:', y_list[t].numpy())
```

Stimuli (8 exemplars x 3 binary dimensions):

```
[[0 0 0]
 [0 0 1]
 [0 1 0]
 [0 1 1]
 [1 0 0]
 [1 0 1]
 [1 1 0]
 [1 1 1]]
```

Type 1 labels: [1. 1. 1. 1. 0. 0. 0. 0.]

Type 2 labels: [1. 1. 0. 0. 0. 0. 1. 1.]

Type 3 labels: [0. 0. 0. 1. 1. 0. 1. 1.]

Type 4 labels: [0. 0. 0. 1. 0. 1. 1. 1.]

Type 5 labels: [0. 0. 0. 1. 1. 1. 1. 0.]

Type 6 labels: [0. 1. 1. 0. 1. 0. 0. 1.]

The ALCOVE model

ALCOVE represents a stimulus x by its similarity to every stored exemplar y_j (here, the 8 SHJ stimuli themselves). Similarity is governed by a city-block distance in psychological space, weighted by a set of learned attention weights α_i , one per stimulus dimension i :

$$\eta(x, y_j) = \exp\left(-c \sum_i \alpha_i |x_i - y_{ji}|\right)$$

Dimensions with a larger attention weight contribute more to similarity, so ALCOVE can learn to "tune out" irrelevant dimensions. The scalar c is a fixed specificity parameter that controls how quickly similarity falls off with distance.

Each exemplar j also has a learned association weight w_j to the category output. The network's similarity between stimulus x and category A is a weighted sum of exemplar activations:

$$\eta(x, A) = \sum_j w_j \eta(x, y_j)$$

Finally, a decision probability is obtained with a sigmoid (this is a simplification of Kruschke's original model, which uses a softmax over multiple categories and also incorporates $\eta(x, B)$, $\eta(x, C)$, etc.; here we have just 2 categories, so one logistic output suffices):

$$p(x \in A) = \text{logistic}(\phi \cdot \eta(x, A))$$

where ϕ is a fixed scaling parameter for the decision.

There are two sets of learnable parameters: the attention weights α_i (one per dimension) and the association weights w_j (one per exemplar). Both are learned by

gradient descent, but they use different learning rates, and the attention weights are constrained to stay non-negative.

Problem 1 (30 points)

Complete the `forward` method of the `ALCOVE` class below. Given a single stimulus `x` (a length-`dim` tensor), compute the probability of responding that `x` is in category `A`.

The constructor is already written for you: it stores the exemplars, initializes the attention weights to be uniform across dimensions, and initializes the association weights to zero.

Hint: `self.exemplars` is `[ne x dim]` and `x` is `[dim]`. You'll want to broadcast `x` against every row of `self.exemplars`, e.g. via `self.exemplars - x.view(1,-1)`, and similarly broadcast `self.attn` (shape `[dim]`) against the result.

```
In [7]: class ALCOVE(nn.Module):

    def __init__(self, exemplars, c=6.5, phi=2.5):
        # Input
        #   exemplars : [ne x dim tensor] rows are the exemplars stored in m
        #   c : specificity of the exemplar similarity gradient
        #   phi : decision temperature
        super().__init__()
        self.ne = exemplars.size(0) # number of exemplars
        self.dim = exemplars.size(1) # number of stimulus dimensions
        self.exemplars = exemplars

        # attention weights start out uniform across dimensions
        self.attn = nn.Parameter(torch.ones(self.dim) / float(self.dim))

        # association weights start out at zero (no basis yet for preferring
        self.w = nn.Linear(self.ne, 1, bias=False)
        self.w.weight = nn.Parameter(torch.zeros(1, self.ne))
        self.c = c
        self.phi = phi

    def forward(self, x):
        # Input
        #   x : [dim tensor] a single stimulus
        # Output
        #   output : [scalar tensor] unnormalized output (before the sigmoid)
        #   prob : [scalar tensor] sigmoid decision probability

        # memory/hidden layer computes the similarity of stimulus x to each
        # YOUR CODE HERE
        hidden = torch.exp(-self.c * torch.sum(self.attn.view(1,-1) * torch.
```

```

        output = self.phi * self.w(hidden).squeeze() # phi * sum_j w_j * h_j
        prob = torch.sigmoid(output) # probability of category A
        return output, prob

```

Run the sanity check below on a freshly initialized network. Since the association weights start at zero, the output should be exactly 0 and the probability exactly 0.5 for every stimulus, regardless of the (uniform, untrained) attention weights.

```

In [8]: net = ALCOVE(X)
        print(net.ne)
        print('output, probability, for each stimulus:')
        for x in X:
            out, prob = net.forward(x)
            print(round(out.item(), 4), round(prob.item(), 4))

```

```

8
output, probability, for each stimulus:
0.0 0.5
0.0 0.5
0.0 0.5
0.0 0.5
0.0 0.5
0.0 0.5
0.0 0.5
0.0 0.5

```

Training ALCOVE

We train ALCOVE with **batch gradient descent**: on each epoch, the network sees all 8 exemplars, and a single optimizer step is taken using the summed loss across all of them. We use the standard **binary cross-entropy** loss on the network's raw output (before the sigmoid), via `torch.nn.BCEWithLogitsLoss` :

$$\ell(\text{output}, \text{target}) = -[\text{target} \cdot \log \text{logistic}(\text{output}) + (1 - \text{target}) \cdot \log(1 - \text{logistic}(\text{output}))]$$

where `target` is 1 for category A and 0 for category B. This is provided for you below, along with a `train` function and an `evaluate` function.

```

In [9]: def update_batch(net, exemplars, targets, loss, optimizer):
        # Update the weights using batch SGD for the entire set of exemplars
        #
        # Input
        #   exemplars: [ne x dim tensor] all stimuli/exemplars in experiment
        #   targets:   [ne tensor] classification targets (1./0.)
        #   loss:      function handle
        #   optimizer : SGD optimizer
        net.zero_grad()
        net.train()
        n_exemplars = exemplars.size(0)
        out = torch.zeros(n_exemplars)
        for j in range(n_exemplars):
            out[j], _ = net.forward(exemplars[j])

```

```

        myloss = loss(out, targets)
        myloss.backward()
        optimizer.step()
        net.attn.data = torch.clamp(net.attn.data, min=0.) # ensure attentio
        return myloss.cpu().item()

def train(exemplars, labels, num_epochs, lr_assoc=0.03, lr_attn=0.0033, track_inc=10, track_out=10):
    # Train an ALCOVE model on one SHJ problem
    #
    # Input
    #   exemplars : [ne x dim tensor] rows are exemplars
    #   labels : [ne tensor] category labels for this problem (1./0.)
    #   num_epochs : number of passes through the exemplar set
    #   lr_assoc : learning rate for association weights
    #   lr_attn : learning rate for attention weights
    #   track_inc : record progress every this many epochs
    # Output
    #   v_epoch, v_prob, v_acc : lists (same length) tracking epoch index,
    #                           probability correct, and percent accuracy
    n_exemplars = exemplars.size(0)
    net = ALCOVE(exemplars)
    optimizer = optim.SGD([
        {'params': net.w.parameters()},
        {'params': [net.attn], 'lr': lr_attn}
    ], lr=lr_assoc)
    loss = torch.nn.BCEWithLogitsLoss(reduction='sum')

    v_epoch = []
    v_prob = []
    v_acc = []
    for epoch in range(1, num_epochs + 1):
        loss_epoch = update_batch(net, exemplars, labels, loss, optimizer)
        if epoch == 1 or epoch % track_inc == 0:
            test_prob, test_acc = evaluate(net, exemplars, labels)
            v_epoch.append(epoch)
            v_prob.append(test_prob)
            v_acc.append(test_acc)

    return net, v_epoch, v_prob, v_acc

def evaluate(net, exemplars, targets):
    # Compute the mean probability of the correct response, and percent accuracy
    #
    # Input
    #   exemplars : [ne x dim tensor] all stimuli in the experiment
    #   targets : [ne tensor] classification targets (1./0.)
    net.eval()
    n_exemplars = exemplars.size(0)
    v_acc = np.zeros(n_exemplars)
    v_prob = np.zeros(n_exemplars)
    for j in range(n_exemplars):
        out, prob = net.forward(exemplars[j])
        out = out.item()
        prob = prob.item()
        if targets[j].item() == CLASS_A:
            v_prob[j] = prob

```

```

        v_acc[j] = out >= 0
    else:
        v_prob[j] = 1 - prob
        v_acc[j] = out < 0
    return np.mean(v_prob), 100. * np.mean(v_acc)

```

Run the cell below to train ALCOVE on all 6 SHJ problem types and plot the resulting learning curves. This should reproduce (approximately) the classic pattern of results from Kruschke (1992): Type I learned fastest, Type VI learned slowest, with Types III–V in between. **Note that this pattern will not be exactly as in the slides, which used Kruschke's original loss function rather than binary cross entropy.**

```

In [10]: num_epochs = 50
list_nets = []
list_trackers = []
for mytype in range(6):
    print('Training on Type ' + str(mytype + 1) + '...')
    net, v_epoch, v_prob, v_acc = train(X, y_list[mytype], num_epochs)
    list_nets.append(net)
    list_trackers.append((v_epoch, 1-np.array(v_prob), v_acc))

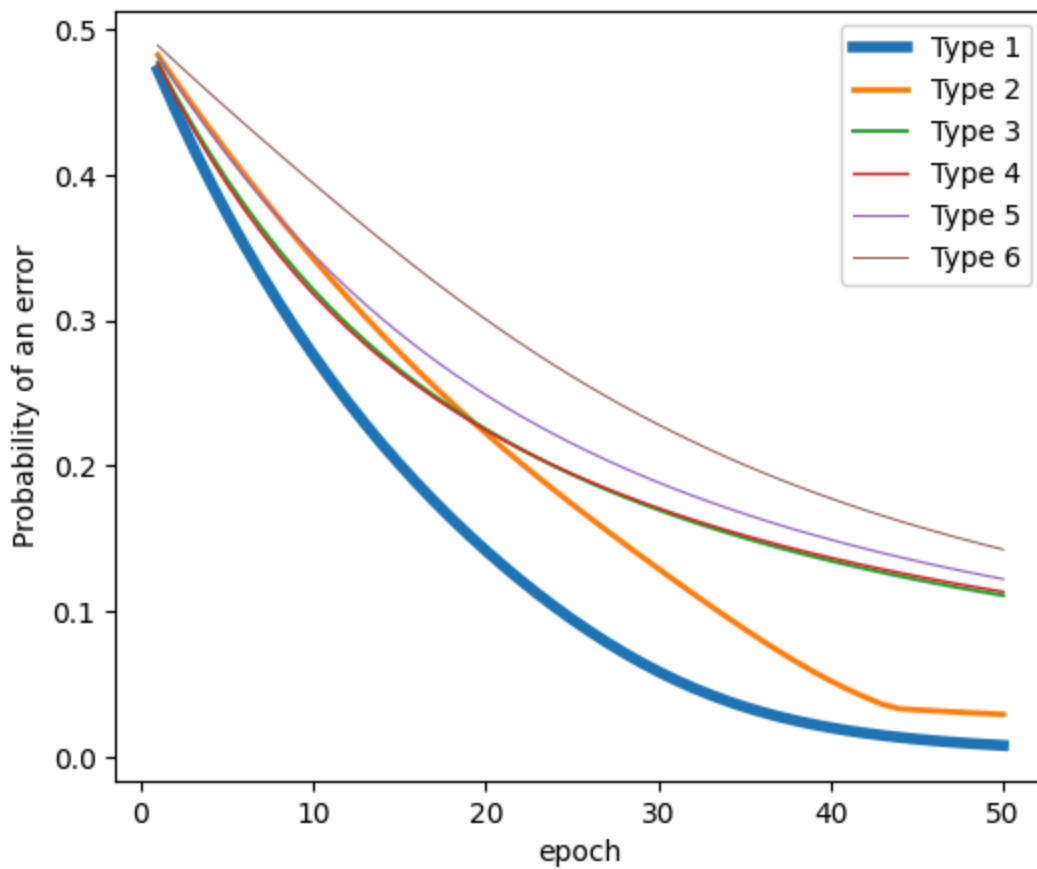
plt.figure(figsize=(6, 5))
for mytype in range(6):
    v_epoch, v_prob, v_acc = list_trackers[mytype]
    plt.plot(v_epoch, v_prob, linewidth=4. / (mytype + 1), label='Type ' + str(mytype + 1))
plt.xlabel('epoch')
plt.ylabel('Probability of an error')
plt.legend()
plt.show()

```

```

Training on Type 1...
Training on Type 2...
Training on Type 3...
Training on Type 4...
Training on Type 5...
Training on Type 6...

```



Here is what my learning curve looked like:

